

Series: Hardening Apache HTTPD in Docker (Part 3 of 4)

Author: H. Brister Wooley

Published: 2026-02-08

Series: Hardening Apache HTTPD in Docker (3/4)

Published Link: <https://github.com/hbwooley/deep-thoughts-and-ramblings/blob/main/2026-02-08-hardening-apache-docker-part3.md>

Welcome to Part 3. We've built a non-root container and locked down `httpd.conf` with default-deny directory rules. The server is quiet, it's contained, and it's not leaking version info.

But here's the thing – attackers don't care about your container architecture. They care about what the browser sees. Every response header is an instruction to the browser. TLS negotiation is a handshake you control (or don't). And the WAF? That's the layer that catches what your code misses.

Today we cover the three layers attackers actually interact with: **security headers** (browser enforcement), **TLS hardening** (transport enforcement), and **ModSecurity** (payload enforcement). Plus the CSP mistake that breaks login pages and how to fix it properly.

Important: In production, these configurations live in `ssl.conf` (the TLS virtual host on port 443), not `httpd.conf`. The `httpd.conf` handles HTTP->HTTPS redirect and baseline directory rules. The `ssl.conf` is where the actual security enforcement happens – headers, TLS, CSP, WAF, and proxy rules.

HTTP Security Headers: The Browser's Instruction Manual

In `ssl.conf` (TLS VirtualHost on port 443)

Header `always set X-XSS-Protection "1; mode=block"`

Header `always set X-Frame-Options SAMEORIGIN`

Header `always set Strict-Transport-Security "max-age=31536000; includeSubDomains; preload"`

Header `always set X-Content-Type-Options "nosniff"`

Header `always set Referrer-Policy "no-referrer-when-downgrade"`

Header `set Cache-Control "no-store, no-cache"`

Header `always edit Set-Cookie ^(.*)$ $1;HttpOnly;Secure;SameSite=strict`

Header	Attack Prevented
X-XSS-Protection: 1; mode=block	Reflected XSS in older browsers

Header	Attack Prevented
X-Frame-Options: SAMEORIGIN	Clickjacking
HSTS: max-age=31536000; includeSubDomains; preload	SSL stripping, MITM downgrade
X-Content-Type-Options: nosniff	MIME-sniffing .jpg as text/html
Referrer-Policy: no-referrer-when-downgrade	Leaks sensitive URLs to HTTP destinations
Cache-Control: no-store, no-cache	Back-button exposure on shared computers
Set-Cookie: ...; HttpOnly; Secure; SameSite=strict	CSRF blocked, no JS access, HTTPS only

The always keyword matters – sets headers even on error responses (4xx, 5xx). Without it, your 500 page leaks without protection.

Content Security Policy: Default Deny, Explicit Allow

```
# In ssl.conf (TLS VirtualHost on port 443)
Header always set Content-Security-Policy "default-src 'self'
https://chat.example.org; \
script-src 'self' 'unsafe-inline' 'unsafe-eval' https://www.google.com ...; \
style-src 'self' 'unsafe-inline' https://chat.example.org
https://fonts.googleapis.com; \
connect-src 'self' https://www.google.com https://chat.example.org; \
img-src 'self' data: https://www.google.com ...; \
frame-src 'unsafe-inline' https://www.google.com https://chat.example.org; \
frame-ancestors 'self' https://chat.example.org; \
font-src 'self' https://fonts.gstatic.com;"
```

default-src 'self' = baseline. Each directive overrides for specific types. **Why enumerate every third-party?** Because default-src 'self' blocks everything else. Forces you to audit every external dependency – a security win by itself.

The 'unsafe-inline' trap: Remove it if you can. Use nonces instead:

```
Header always set Content-Security-Policy "script-src 'self' 'nonce-
%{UNIQUE_ID}e'; ..."
```

Your app renders `<script nonce="<%= request.attr('UNIQUE_ID') %>">....` More work, actual security.

Permissions Policy: Disable What You Don't Use

```
# In ssl.conf (TLS VirtualHost on port 443)
Header set Permissions-Policy: "accelerometer=(), camera=(),
geolocation=(self), \
fullscreen=(self), microphone=(), payment=(), usb=(), web-share=()"
```

Empty list () = disabled. (self) = same-origin only. Your portal doesn't use the camera. If an attacker injects a script, the browser refuses the API call. XSS blast radius shrinks.

TLS Hardening: Why TLS 1.2 Only, Why No CBC

```
# In ssl.conf (TLS VirtualHost on port 443)
SSLProtocol -ALL +TLSv1.2
SSLHonorCipherOrder on
SSLCipherSuite HIGH:!aNULL:!MD5:!3DES:!RC4:!CBC:!AES128-SHA:!AES128-
SHA256:!AES256-SHA:!AES256-SHA256:!CAMELLIA128-SHA:!CAMELLIA128-
SHA256:!CAMELLIA256-SHA:!CAMELLIA256-SHA256:!DHE-RSA-AES128-SHA:!DHE-RSA-
AES128-SHA256:!DHE-RSA-AES256-SHA:!DHE-RSA-AES256-SHA256:!DHE-RSA-
CAMELLIA128-SHA:!DHE-RSA-CAMELLIA128-SHA256:!DHE-RSA-CAMELLIA256-SHA:!DHE-
RSA-CAMELLIA256-SHA256:!ECDHE-RSA-AES128-SHA:!ECDHE-RSA-AES128-SHA256:!ECDHE-
RSA-AES256-SHA:!ECDHE-RSA-AES256-SHA384:!ECDHE-RSA-CAMELLIA128-SHA256:!ECDHE-
RSA-CAMELLIA256-SHA384
```

Translation:

Setting	Meaning
SSLProtocol -ALL +TLSv1.2	Only TLS 1.2. No 1.0/1.1 (broken), no 1.3 (baseline hasn't approved)
SSLHonorCipherOrder on	Server picks cipher, not client. Prevents downgrade
SSLCipherSuite HIGH:!aNULL:!MD5:!3DES:!RC4:!CBC:...	Removes anonymous auth, MD5, 3DES, RC4, all CBC , 128-bit, SHA-1, CAMELLIA, weak DHE/ECDHE. Left: AES-256-GCM and CHACHA20-POLY1305 – AEAD, no padding oracle

Why no CBC? Padding oracle attacks (Lucky Thirteen, POODLE). AEAD authenticates *before* decrypting – no padding, no oracle.

Why TLS 1.2 only? Compliance baseline. When it updates, change one line.

ModSecurity WAF: CRS with Surgical Exclusions

```
# In ssl.conf (TLS VirtualHost on port 443)
<IfModule security2_module>
    SecRuleEngine On
    SecRequestBodyAccess On
    SecRequestBodyLimit 13107200
    SecAuditEngine RelevantOnly
    SecAuditLogRelevantStatus "^(?:5|4(?:!04))"
    SecAuditLogFormat JSON
    SecAuditLogType Concurrent
    SecAuditLogStorageDir /var/log/httpd/modsec_audit
    SecAuditLog /var/log/httpd/modsec_audit.log

    Include /usr/local/apache2/conf/modsecurity.d/coreruleset-3.3.4/crs-
setup.conf
    Include /usr/local/apache2/conf/modsecurity.d/coreruleset-
3.3.4/rules/*.conf
    Include /usr/local/apache2/conf/modsecurity.d/local_rules/*.conf
</IfModule>
```

Key settings:

Setting	Purpose
SecRuleEngine On	Blocking mode. DetectionOnly = logs only (tuning only)
SecRequestBodyAccess On	Inspects POST/PUT bodies (JSON, XML, form)
SecAuditEngine RelevantOnly	Logs only transactions with triggered rules or errors (4xx/5xx except 404). Cuts volume 90%+
SecAuditLogFormat JSON	Structured logs for SIEM/Splunk/Elastic
SecAuditLogType Concurrent	Each transaction to own file. No lock contention
Include ... local_rules/*.conf	Your overrides. Never edit CRS directly. Loads last, wins

Per-Location Exclusions: Surgical, Not Nuclear

```
<LocationMatch "^/server/api/system/scripts/export">
    <IfModule mod_security2.c>
        SecRuleEngine Off
        SecAuditEngine Off
    </IfModule>
</LocationMatch>

<LocationMatch "^/server/api/core/bundles">
```

```
<IfModule mod_security2.c>
  SecRuleRemoveById 920350 949110 920420 980130 200003
  SecRequestBodyLimit 16106127360
</IfModule>
</LocationMatch>
```

Why this approach:

- **metadata-export** – Huge XML/CSV exports trigger 50 CRS rules. Turn off engine for *this one endpoint*. Internal, authenticated, rate-limited – acceptable risk.
- **bundles** – Legitimate 16 GB uploads. Don't turn off engine. Surgically remove 5 false-positive rules:
 - 920350 – Host header anomalies (large uploads trigger buffer logic)
 - 949110 – XSS in body (base64 blobs look like scripts)
 - 920420 – SQLi in args (binary data matches SQL patterns)
 - 980130 – RFI detection (URLs in multipart parts)
 - 200003 – Multipart boundary validation (large parts look malformed)
 - 16 GB limit *only* for this endpoint.

Why exceptions are sometimes necessary:

The CRS is designed to catch *malicious* patterns – but legitimate applications sometimes generate traffic that *looks* malicious to a pattern matcher. Base64-encoded binary blobs look like obfuscated payloads. Large multipart uploads trigger buffer overflow heuristics. XML exports with inline data match SQL injection patterns.

You have two choices: break the application, or scope an exception.

Breaking the application means developers work around you – they disable the WAF entirely, move the endpoint outside the proxy, or (worst case) push vulnerable code because “the WAF blocks it anyway.”

Scoped exceptions keep the WAF active for 99.9% of traffic while acknowledging reality: **your job is to secure the application, not prevent it from working.**

The key is *how* you scope it: - **Exact LocationMatch** – only this path, not /bundles/* or /api/** - **Specific rule IDs** – only the 5 rules causing false positives, not the whole engine - **Documented rationale** – each SecRuleRemoveById has a comment explaining *why* and *what mitigates the risk* (internal only, authenticated, rate-limited, etc.) - **Audit trail** – the config *is* the documentation. Auditors see exactly what was excluded and why.

The rule: Never disable SecRuleEngine globally. Never disable a rule globally. Scope to exact LocationMatch.

What This Means for Your Strategy

Here's the strategic payoff: when you configure headers, TLS, and WAF together, you create a **unified enforcement plane**. The browser enforces CSP/HSTS. The transport enforces TLS 1.2 + AEAD. The WAF enforces payload rules. An attacker has to bypass all three simultaneously – and they're independent controls.

That's not just "defense in depth." That's **defense in breadth** – different threat models, different failure modes, different attackers. You're not layering the same defense three times. You're covering three different attack surfaces.

And those surgical exclusions? That's how you keep developers happy *and* security intact. "No WAF for this endpoint" is a conversation. "SecRuleRemoveById 920350 949110 for /bundles only" is a policy decision with an audit trail.

What's Coming Next

Part 4 covers docker-compose.yml runtime hardening (capabilities, networks, volumes), rewrite rules that block AI bots and integrate with fail2ban, and the 10 verification commands for every deploy.

Plus the RewriteMap trick that makes fail2ban integration atomic – no partial reads, no race conditions.

Read Part 4 ->

Appendix: CompTIA Security+ SY0-701 CEU Mapping

This appendix maps each technique in this post to CompTIA Security+ exam objectives for CEU submission.

Technique Implemented	Security+ Domain	SY0-701 Objective	Rationale
HSTS with preload	Domain 3: Security Architecture	3.5: Implement PKI/certificate management	Forces HTTPS, prevents SSL stripping
X-Frame-Options: SAMEORIGIN	Domain 2: Security Architecture	2.3: Implement application security	Clickjacking prevention
X-Content-Type-Options: nosniff	Domain 2: Security Architecture	2.3: Implement application security	MIME type confusion prevention
CSP with default-src 'self' + explicit allowlists	Domain 2: Security Architecture	2.3: Implement application security	XSS mitigation via script/style source control
CSP nonces instead of 'unsafe-inline'	Domain 2: Security Architecture	2.3: Implement application security	Cryptographic nonce validation prevents script injection
Permissions-Policy disabling unused APIs	Domain 2: Security Architecture	2.3: Implement application security	Reduces XSS blast radius (camera, mic, geolocation)
TLS 1.2 only + AES-256-GCM/CHACHA20-POLY1305	Domain 3: Security Architecture	3.5: Implement PKI/certificate management	AEAD ciphers, no padding oracle attacks
SSLHonorCipherOrder on	Domain 3: Security Architecture	3.5: Implement PKI/certificate	Server-side cipher

Technique Implemented	Security+ Domain	SY0-701 Objective	Rationale
		management	selection prevents downgrade
ModSecurity CRS 3.3.4 with OWASP rules	Domain 2: Security Architecture	2.3: Implement application security	Web Application Firewall for SQLi, XSS, RFI, RCE
SecAuditEngine RelevantOnly + JSON	Domain 4: Security Operations	4.1: Apply security monitoring concepts	Structured audit logging for SIEM ingestion
Surgical LocationMatch exclusions	Domain 3: Security Architecture	3.2: Implement secure system design	Scoped exception handling with audit trail
SecRuleRemoveById with documented rationale	Domain 4: Security Operations	4.2: Apply incident response procedures	Documented risk acceptance process

Primary Domain Coverage: Domain 2 (Architecture) – ~45% | Domain 3 (Architecture & Engineering) – ~40% | Domain 4 (Operations) – ~15%

Total Security+ Mapping: >=50% of content maps to SY0-701 objectives